RAG and ChromaDB: Practical Guide

How retrieval-augmented generation works, how ChromaDB query output is shaped, and how to filter results.

The RAG pipeline

Retrieval-augmented generation (RAG) is a pattern in which an application retrieves external material and includes that material as context for a language model. A basic pipeline is load documents → extract text → split into chunks → create embeddings → store vectors → search using a user query → pass selected chunks to an LLM → generate an answer.

Retrieval helps a model draw on external documents, but does not guarantee correctness. Good applications preserve source information so users can verify where claims came from.

Document ingestion and chunking

First extract readable text from PDFs. A PDF containing selectable text is generally easier to ingest than an image-only scanned PDF, which may require OCR. Split long documents into chunks based on sensible semantic boundaries such as sections or paragraphs.

Very small chunks can lose context; very large chunks can mix topics and consume the prompt budget. Chunk overlap can preserve context across boundaries, but it also creates redundant records. Attach metadata such as source_file, section, and page_number where available.

Embeddings and ChromaDB collections

A vector database stores numeric embeddings and links them to content and identifiers. A ChromaDB collection can store document IDs, text, embeddings, and metadata. Queries return nearest items under the collection's configured distance metric. Use the same compatible embedding approach for indexed text and new queries.

ChromaDB supports different distance metrics, including l2, cosine, and ip. Check the collection configuration before interpreting any numeric distances. A small cosine distance indicates greater cosine similarity; raw scores from different metrics should not be treated as interchangeable.

Why query results contain nested lists

ChromaDB query responses organize nearest results by input query. For one input query, an ID field might look like [["chunk-1", "chunk-2"]]. The outer list is for the query; the inner list holds that query's matches. Therefore results["ids"][0] extracts a flat list of matching IDs.

If no matches are returned for a query, the shape may be [[]]. In that case results["ids"][0] is an empty list, and range(len(ids)) iterates zero times. Index 0 assumes you intend to process only the first query in the batch.

An index-based result-processing example

When a ChromaDB query includes documents, metadata, and distances, each aligned inner list describes the same ranked results. The following pseudocode illustrates straightforward iteration:

```
ids = results["ids"][0]
docs = results["documents"][0]
```

```
metas = results["metadatas"][0]
dists = results["distances"][0]

for i in range(len(ids)):
    doc_id = ids[i]
    content = docs[i]
    metadata = metas[i]
    distance = dists[i]
```

The enumerate(zip(ids, docs, metas, dists)) form is an alternative that associates aligned items and supplies their index. Unlike direct indexing, zip stops at the shortest input iterable by default. Results should normally be aligned, but validate unexpected upstream data.

Converting cosine distance to similarity

For a ChromaDB collection explicitly configured for cosine distance, the usual relationship is cosine similarity = 1 − cosine distance. For example, if distance is 0.25, cosine similarity is 0.75. A filter with score_threshold = 0.70 would keep that example.

Do not assume the formula produces a cosine similarity score for l2 or ip distance. Cosine similarity mathematically ranges from -1 to 1, so it is not always a percentage and not universally bounded between 0 and 1. A similarity threshold is a tunable relevance heuristic, not a factual-accuracy guarantee.

Retrieval quality and useful debugging checks

- If no documents are retrieved, inspect whether text was extracted, vectors were inserted, the collection exists, and the threshold is too strict.
- If retrieval finds the wrong sections, examine chunk sizes, embedding model compatibility, metadata filters, and the actual query wording.
- If answers sound confident but are unsupported, instruct the model to cite retrieved sources, to separate facts from inference, and to say when evidence is missing.
- For evaluation, create test questions with known supporting passages. Track whether the expected chunk appears in top-k and whether final answers are grounded in that chunk.

Questions this document can answer

Q: Why is results["ids"] shaped like a list of lists?
A: ChromaDB groups matching IDs per input query; the outer list is query-level and the inner list holds matches.

Q: What does [[]] mean?
A: The query-level result exists but its list of matching IDs is empty.

Q: When is similarity_score = 1 - distance correct?
A: For results measured with cosine distance; confirm your collection uses the cosine metric before applying it.

Q: How can I troubleshoot an empty retrieval result?
A: Check PDF text extraction, indexed chunks, collection configuration, metadata filters, and the similarity threshold.